SolvaRisk — Automatisation IA pour Solvabilité II

Rapport technique — Advanced Deep Learning, projet final *AI Automation* (5DS, Esprit 2026-2027)

Équipe SolvaRisk | dépôt : `Solva/` | bibliothèque de prompts : `docs/prompt_library.md` | évaluation du retrieval : `rag_service/eval/`

1 Cas d'usage et vue d'ensemble

Problème. Un organisme d'assurance soumis à Solvabilité II doit produire chaque trimestre ses états réglementaires (QRT), calculer son capital de solvabilité requis (SCR) et son ratio de couverture, auditer la qualité des données, rédiger un diagnostic ORSA et vérifier des règles réglementaires. Le travail est répétitif, chiffré, et une erreur de chiffre est inacceptable : c'est l'argument central de nos choix d'architecture (*le modèle de langage ne calcule rien*).

Système. SolvaRisk est une application Next.js reliée à un service Python (FastAPI) et à un workflow n8n. Elle propose cinq usages : (1) import d'un QRT (Excel ou PDF), audit, calcul du SCR, diagnostic de l'analyste et rapport PDF en un clic ; (2) un copilote RAG sur le corpus réglementaire ; (3) un agent « Expert EIOPA & QRT » qui vérifie les limites de fonds propres et génère un Excel QRT ; (4) un agent « Climate & ESG » (test de résistance ORSA aux inondations) ; (5) l'historique des rapports.

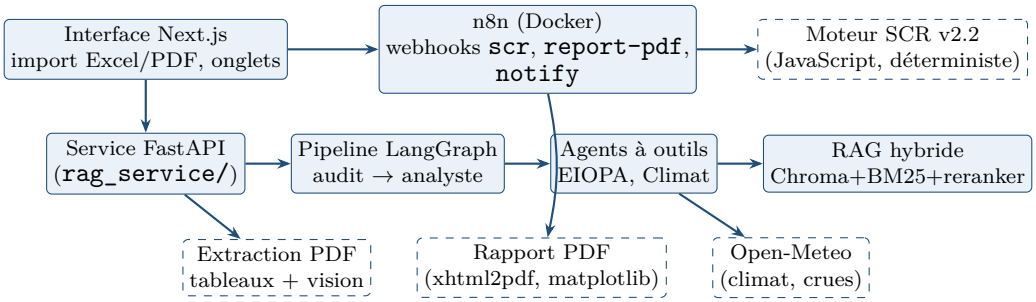


Figure 1 – Architecture. Cadres pleins : composants à LLM ou à orchestration ; cadres en pointillés : composants déterministes.

2 Plateforme d'automatisation (n8n) — ce qui tourne où

Le workflow n8n exporté (`n8n/solvarisk-workflow.json`, 15 nœuds, généré par `build-workflow.cjs`) expose trois webhooks : `scr` (nœud Code « Moteur SCR v2.2 » : agrégation par matrices de corrélation, diversification, absorption par les provisions techniques et impôts différés, éligibilité des fonds propres par Tier, contrôles de cohérence), `report-pdf` (appel du conteneur Python `report-pdf` qui rend le PDF) et `notify` (nœud Switch puis Gmail pour les alertes et l'avis de rapport prêt). Le moteur SCR est volontairement un nœud de code déterministe : la plateforme affiche *exactement* sa sortie.

Écart assumé par rapport à l'exigence 1. La cascade « Calculer → SCR → analyste → rapport PDF → notification » est déclenchée de bout en bout par un seul clic, mais son orchestration est portée par l'application et le service FastAPI (graphe LangGraph), pas par un unique workflow n8n ; n8n porte le calcul SCR, la génération du rapport et la notification. Nous l'assumons : le premier essai d'agents LLM dans n8n a échoué (voir §8) et le graphe Python est testable hors ligne. Une version où n8n appelle `/api/pipeline/audit-and-risk` comme un nœud HTTP est la suite naturelle.

3 Ingénierie de prompts

La bibliothèque versionnée (`docs/prompt_library.md`, 7 prompts) documente pour chacun le rôle, la technique, les échecs observés et le correctif. Techniques utilisées : *role prompting* (actuaire senior), *sortie structurée* (schémas Pydantic, `response_schema` Gemini), *grounding avec citations* (`[n]` et refus

Webhook	Nœuds	Comportement
`scr`	Webhook → Moteur SCR v2.2 → Réponse (ou Erreur 422)	valide les feuilles QRT, calcule SCR, MCR, ratio, diversification, éligibilité par Tier, contrôles de cohérence ; erreur 422 explicite si une entrée manque
`report-pdf`	Webhook → Python PDF Generator → Réponse (ou Erreur 502)	transmet le diagnostic au conteneur `report-pdf` (port 8002) et renvoie le PDF binaire
`notify`	Webhook → Switch → Mail alerte / Mail rapport → Gmail	e-mail d'alerte ou avis de rapport prêt ; type inconnu ignoré sans erreur

Table 1 – Les trois flux du workflow n8n exporté.

littéral), *agent à outils* (procédure obligatoire), *garde-fous* (vérification en code de ce que le modèle déclare).

Prompt	Échec observé (avant)	Correctif (après)
Analyste (P3)	Français dégradé : apostrophes et accents supprimés	règle 7 + nouvel essai si texte dégradé + réparation des élisions en code
Analyste (P3)	PDF de synthèse : SCR recalculé faux (10 075,96) au lieu des chiffres publiés	règle 9 + mode « synthèse » + garde en code
Analyste (P3)	Conseil générique « améliorer la qualité des données »	règle 6 : citer les constats précis de l'audit
Agent EIOPA (P6)	« Règle confirmée » déduite d'un score de recherche (un passage sur le SCR « confirmait » une limite de Tier)	le modèle *déclare* `rule_confirmed`+preuves, le code les vérifie (pertinence $\geq 0{,}30$)
Agent EIOPA (P6)	« ÉCART » affiché pour un plafond dépassé (fausse infraction)	statut plafonné défini dans le prompt
Agent EIOPA (P6)	372,085 arrondi à 372,08 (le moteur n8n donne 372,09)	arrondi « demi vers le haut » identique à `Math.round`

Table 2 – Extrait des itérations avant/après (détail complet dans la bibliothèque).

Exemple détaillé d'itération (P6, confirmation de règle). L'agent EIOPA devait dire si la base documentaire *confirme* les limites de Tier. En v1, le champ reposait sur le score de recherche : un passage sur le SCR en général, bien classé, suffisait à afficher « règle confirmée ». En v2, la responsabilité est partagée :

```
- Prompt   : "Un passage ne confirme une regle que s'il l'ENONCE (les seuils eux-memes) ...
    declare ton jugement dans
              check_tier_limits (rule_confirmed, evidence_ids) ; s'il est negatif, ecris : regle
                  non confirmee par la base."
- Code     : rule_confirmed = declare ET au moins un evidence_id existe parmi les passages
    retrouves avec pertinence >= 0,30
- Resultat : si aucun passage ne fournit les seuils (cas du corpus actuel, sans les regles de
    Tier) -> rule_confirmed = false
```

Le principe, repris dans tous les agents : *ce que le modèle affirme est vérifié par le code avant d'être affiché comme un fait.*

Ce qui n'a pas été itéré. Les prompts RAG (réécriture, réponse), d'extraction par vision et de l'agent climat sont en v1 : nous n'avons pas observé d'échec qui justifie une itération, et nous n'avons pas mesuré l'apport de la réécriture de requête. La bibliothèque le dit explicitement.

ID	Prompt	Technique	Version
P1	Réécriture de la question (RAG)	sortie JSON, acronymes développés	v1
P2	Réponse citée (RAG)	grounding, citations `[n]`, refus littéral	v1
P3	Analyste de risques	rôle, 9 règles, sortie structurée, garde-fous en code	v3
P4	Extraction PDF par vision	sortie structurée, « `null` plutôt que deviner »	v1
P5	Extraction d'un PDF de synthèse	sortie structurée, périmètre explicite	v2
P6	Agent Expert EIOPA & QRT	agent à outils, procédure obligatoire, honnêteté	v3
P7	Agent Climate & ESG	agent à outils, chiffres interdits au modèle	v1.1

Table 3 – Les sept prompts de la bibliothèque et leur version courante.

4 RAG : corpus, découpage, retrieval, évaluation

4.1 Corpus

Trois documents de cours en PDF, en français : *Solvabilité 2 version courte* (12 p.), *Pilier 1 et ORSA* (73 p.) et *Formation Solvabilité 2 – IRA* (106 p.), soit 191 pages et environ 73 000 caractères de texte extrait. Choix : ils couvrent exactement le périmètre du projet (piliers, SCR, MCR, marge de risque, ORSA, QRT) et sont la référence des étudiants. **Limites constatées** : (i) la formation IRA est en grande partie composée de diapositives-images (106 pages pour 25 600 caractères) : le texte est absent, une OCR serait nécessaire ; (ii) le corpus ne contient *pas* les règles de classement par Tier (règlement délégué 2015/35) : l'agent EIOPA répond alors « règle non confirmée par la base documentaire » au lieu d'inventer ; (iii) la diapositive 81 de la formation donne un ratio de couverture inversé (SCR / fonds propres) que le RAG reproduirait fidèlement.

4.2 Découpage, embeddings, index

Stratégie de production (`build_chunks`) : découpage par *lignes puis phrases*, taille cible 1 100 caractères, recouvrement 200 (environ 18 %), coupure forcée au changement de page lorsque le chunk est rempli à moitié. Justification : 1 100 caractères ≈ 250–300 jetons, sous la limite de 512 jetons de l'encodeur ; les diapositives sont des listes courtes qu'il faut regrouper sans couper une formule ; le recouvrement évite de séparer une définition de son exemple. Résultat : 96 chunks (7 + 53 + 36). Chaque chunk est indexé avec son titre de document et ses pages.
Embeddings : `multilingual-e5-base` (multilingue, français, préfixes `query:`/`passage:`). **Magasin** : ChromaDB persistant, distance cosinus (HNSW), index incrémental par empreinte de fichier.
Retrieval : recherche dense + BM25 (tokenisation sans accents, mots vides et pluriels retirés), fusion *Reciprocal Rank Fusion* ($k = 60$, 30 candidats, 20 fusionnés), reranking par `bge-reranker-v2-m3`, top-5, seuil de refus 0,05. La requête est d'abord réécrite par le LLM (forme autonome + 2 variantes avec acronymes développés). Sous le seuil, le LLM n'est pas appelé.

4.3 Évaluation du retrieval et comparaison de découpages

Protocole (`rag_service/eval/retrieval_eval.py`, reproductible) : 13 questions dont la réponse est dans le corpus et 2 hors corpus ; vérité terrain = page(s) du PDF contenant la réponse, lue à la main (indépendante du découpage) ; un passage est pertinent s'il est du bon document et recoupe les pages attendues ; requêtes brutes, sans réécriture LLM. Trois découpages sont comparés : A (production, 1 100/200), B (petits chunks 400/80), C (une page = un chunk, structure du document), chacun en quatre modes (dense, BM25, hybride, hybride + reranker). **Questions hors corpus.** Meilleur score du reranker : 0,343 à 1,000 sur les questions du corpus (médiane 0,973), contre 0,147, 0,020 sur les deux questions hors corpus (limites de Tier ; taux directeur de la BCE), pour un seuil de refus de 0,05. **Ce que montrent les résultats.**

— **Le reranker est l'élément qui apporte le plus.** Avec le découpage de production (A), le MRR passe de 0,68 (hybride sans reranker) à 1,00 : le bon passage est en tête pour les 13 questions. Le gain est aussi net pour B (0,74 → 0,92) mais nul pour C (0,87 → 0,88), dont les chunks sont

Découpage / mode	hit@1	hit@3	hit@5	MRR
A. 1100/200 — 96 chunks, 785 car. en moyenne				
dense	69 %	77 %	85 %	0,75
BM25	31 %	62 %	92 %	0,53
hybride (RRF)	54 %	85 %	85 %	0,68
hybride + reranker	100 %	100 %	100 %	1,00
B. 400/80 — 236 chunks, 331 car. en moyenne				
dense	46 %	85 %	92 %	0,65
BM25	38 %	77 %	85 %	0,58
hybride (RRF)	62 %	92 %	92 %	0,74
hybride + reranker	85 %	100 %	100 %	0,92
C. page — 191 chunks, 382 car. en moyenne				
dense	54 %	77 %	92 %	0,66
BM25	46 %	69 %	69 %	0,56
hybride (RRF)	77 %	100 %	100 %	0,87
hybride + reranker	77 %	100 %	100 %	0,88

Table 4 – Qualité du retrieval (13 questions du corpus, top-5, requêtes brutes).

déjà des pages entières.

— **Sans reranker, la fusion hybride n'améliore pas toujours la recherche dense.** Sur A, l'hybride (MRR 0,68) fait moins bien que le dense seul (0,75) : BM25 est faible sur ce corpus (hit@1 de 31 %), probablement parce que les diapositives sont des listes courtes avec peu de termes discriminants ; la fusion RRF dilue alors le bon classement du dense. C'est une hypothèse, que nous n'avons pas testée (pondération de la fusion).

— **Comparaison des découpages.** *Avant* reranking, la page entière (C) est la meilleure stratégie hybride (MRR 0,87 contre 0,74 pour B et 0,68 pour A) : une page de diapositive est une unité de sens. *Après* reranking, c'est le découpage A (1 100/200) qui est le meilleur (hit@1 100 %, contre 85 % pour B et 77 % pour C), car il regroupe plusieurs diapositives voisines qui répondent ensemble à une question. Nous gardons donc A en production.

— **Seuil de refus.** Le meilleur score du reranker vaut au minimum 0,343 sur les questions du corpus. La question hors corpus sur le taux directeur de la BCE obtient 0,020 (rejetée avant tout appel au LLM) ; en revanche la question sur les *limites de Tier* obtient 0,147, *au-dessus* du seuil actuel de 0,05 : un passage sur les fonds propres est transmis au LLM, qui doit alors refuser grâce à la règle 4 du prompt P2. Un seuil entre 0,15 et 0,34 rejetterait cette question sans perdre aucune des 13 questions du corpus ; nous ne l'avons pas appliqué, car 13 questions ne suffisent pas à le justifier.

Limites de cette évaluation (à présenter comme telles) : 13 questions seulement (une question vaut ≈ 8 points de pourcentage, donc les écarts de découpage ne sont pas statistiquement significatifs) ; vérité terrain *par page*, ce qui avantage légèrement les chunks qui couvrent plusieurs pages ; requêtes brutes (l'apport de la réécriture LLM n'est pas mesuré) ; la qualité de la réponse générée et la fidélité des citations ne sont pas évaluées ; un seul modèle d'embedding. Le script et les résultats bruts sont dans `rag_service/eval/` (`results.json`, `retrieval_evaluation.md`).

5 Architecture multi-agents

5.1 Agents, outils, flux d'information

Pipeline séquentiel à branchement conditionnel (LangGraph `StateGraph`) : `data_quality` → (`risk_analyst` si les données sont exploitables ; `blocked_alert` puis fin si une anomalie critique les rend inexploitables). L'état partagé est un objet Pydantic ; le statut final est completed, blocked ou analysis_failed (audit fait, LLM indisponible).

Agent	Nature	Outils / sortie	Entrée → sortie
Audit qualité	règles déterministes (pas de LLM)	contrôles de complétude, cohérence, doublons ; statut passed/warning/blocked	positions → rapport d'audit
Analyste de risques	LLM (Gemini), sortie structurée	indicateurs précalculés en code ; diagnostic, ORSA, atténuation, garde-fous	SCR + audit → diagnostic
Moteur SCR	nœud JS n8n, déterministe	matrices de corrélation, éligibilité Tier	feuilles QRT → SCR, ratio
Expert EIOPA & QRT	LLM à outils (ReAct)	`search_eiopa_vector_db` (RAG), `get_own_funds_portfolio`, `check_tier_limits`, `generate_excel_qrt`	question + fonds propres → verdict, sources, Excel
Climate & ESG	LLM à outils (ReAct)	`query_portfolio_database` (SQL lecture seule), `fetch_geospatial_weather_api` (Open-Meteo), `compute_climate_stress`	question + SCR Non-Vie → % menacé
Copilote	RAG (§4)	réécriture, retrieval hybride, réponse citée	question → réponse + sources

Table 5 – Les agents, leurs outils et leur rôle.

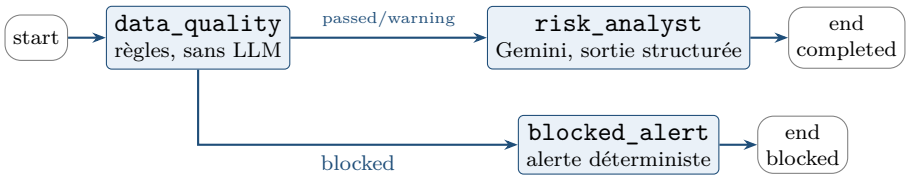


Figure 2 – Graphe LangGraph du pipeline d'audit et de diagnostic. Si le LLM est indisponible, l'audit est tout de même renvoyé (analysis_failed).

Trace réelle de l'agent climat (SCR Non-Vie 163,11 M€, Gemini) : `query_portfolio_database` (lecture des villes et valeurs assurées) → `fetch_geospatial_weather_api` (15 zones en une liste) → `compute_climate_stress` ; la réponse rédigée reprend les pourcentages calculés par le code. Pour l'agent EIOPA, la trace attendue est `search` → `get_own_funds_portfolio` → `check_tier_limits` → `generate_excel_qrt`.

5.2 Pourquoi cette architecture, et pas une autre

— **Un appel monolithique** (un prompt qui lit le QRT, calcule le SCR et rédige) serait pire : le modèle ferait des calculs, non reproductibles ni auditables ; une anomalie de données ne pourrait pas être *bloquante* avant tout appel au LLM (coût et risque de diagnostic fondé sur des données fausses).

— **Chaîne séquentielle avec porte** pour le pipeline : l'ordre est fixe (on n'analyse pas ce qu'on n'a pas validé) et le branchement est une simple condition ; un orchestrateur LLM ajouterait de la latence, du coût et de l'imprévisibilité pour une décision que le code prend mieux.

— **Agents à outils** pour EIOPA et Climat, où l'*ordre* des actions dépend de la question (chercher la règle, lire les données, calculer, éventuellement générer un fichier ; interroger la base puis chaque zone). Garde-fou commun : *le modèle choisit et enchaîne les outils, mais ne saisit jamais un chiffre* ; sources, éligibilité, fichier et pourcentages viennent de l'état des outils. Si tous les modèles échouent, le calcul déterministe est renvoyé (llm_unavailable).

— **Hiérarchique ou maître–ouvriers** ? Rejeté : seulement deux agents LLM par flux, sans sous-tâches parallèles ; la complexité ne se justifie pas (consigne du sujet : une architecture simple et défendable).

Robustesse. Rotation de clés API et de modèles (3 modèles × les clés configurées, période de repos de 10 minutes après un 429), nouvel essai, gestion explicite des erreurs jusqu'à l'interface (jamais

de résultat inventé), 131 tests automatisés hors ligne (LLM simulé, API simulées).

6 Multimodalité

Entrée : PDF avec mise en page. Le PDF d'un QRT ou d'un rapport prudentiel est converti vers *la même structure* que l'import Excel (feuilles de lignes), de sorte que l'audit, le moteur SCR, l'analyste et le rapport restent inchangés. Trois voies : (1) **tableaux** d'un PDF texte lus avec PyMuPDF — le texte de chaque cellule est reconstruit à partir des mots positionnés, car l'extraction native des tableaux produisait un texte corrompu ; les tableaux coupés sur deux pages et une ligne débordant sur la page suivante sont recollés ; (2) **vision** (Gemini, PDF scanné ou sans tableau détectable, sortie structurée, valeurs illisibles à `null`) ; (3) **synthèse** (rapport qui publie des résultats : on relève les chiffres publiés et on ne recalcule rien). La voie vision est marquée « à vérifier ».

Sortie : PDF et Excel. Le rapport d'audit est généré en PDF (gabarits Jinja2, graphiques matplotlib, xhtml2pdf) et l'agent EIOPA produit un classeur Excel au format exact des QRT de référence (test d'aller-retour cellule à cellule, et recalcul par le vrai moteur n8n).

Apport. Les assureurs reçoivent des PDF, pas des tableurs : la modalité supprime la ressaisie et l'erreur associée. *Limite* : sur le PDF « Rapport Prudentiel – Assureur Mixte Solide » la première version recalculait un SCR faux ; la voie « synthèse » et ses contrôles de cohérence ont corrigé le cas, mais l'OCR de documents très dégradés n'est pas évaluée.

7 Climate & ESG Stress Tester

Test de résistance ORSA aux inondations. Pour chaque zone (cellule de 0,1°), l'API Open-Meteo fournit 15 ans de précipitations d'un modèle climatique (EC_Earth3P_HR) et les débits de rivières ; la probabilité annuelle p d'un jour de pluie ≥ 40 mm (seuil supposé) est relevée de 25 % si les rivières sont déjà très hautes, puis $P_N = 1 - (1-p)^N$ sur l'horizon N. La part du SCR Non-Vie menacée est $\sum_z V_z P_{N,z} / \sum_z V_z$ (V_z : valeur assurée avec garantie inondation), la perte attendue $= 2\,\% \times \sum_z V_z P_{N,z}$ (taux supposé). Exemple réel, SCR Non-Vie 193,11 M€, portefeuille de démonstration : 46,7 % à 3 ans, 51,6 % à 5 ans, 57,5 % à 10 ans ; la part en zones à risque élevé reste 34,8 %. **Ce n'est pas un modèle de catastrophe calibré** (un seul modèle climatique, seuil et taux de dommage supposés), et le résultat ne dépend pas du SCR (seuls les montants en M€ en dépendent).

8 Tests, échecs et limites

Ce qui n'a pas marché (et pourquoi).

— *Agent LLM dans n8n* : erreurs « fetch failed » (réseau du conteneur), modèle retiré (404), surcharge 503, conflit de chemins de webhooks après import ; d'où le passage des agents LLM en Python (testables, rejouables), n8n gardant le déterministe.

— *Quotas Gemini (429)* : les agents tombaient en panne quand un modèle était épuisé ; correctif : rotation de clés/modèles et repli déterministe.

— *Sensibilité du stress test climatique* : la première version (classes figées) donnait toujours 20 % ; refaite en probabilités sur l'horizon. Un cache disque pollué par un de nos tests a faussé un run ; tests isolés depuis. Une API gratuite trop sollicitée (429) imposait de raccourcir la fenêtre de 25 à 15 ans.

— *Moteur SCR* : ordre des modules décalé entre le vecteur et la matrice BSCR (corrigé), actifs incorporels mal agrégés dans le générateur de données de test (corrigé).

— *Service obsolète* : des résultats « qui ne changent pas » venaient d'un serveur non redémarré après modification du code.

Tests. 131 tests automatisés sans réseau (modèle de chat scripté qui appelle réellement les outils, API simulées) :

Suite	Tests	Ce qui est vérifié
`pipeline/tests`	55	audit, branchement, garde-fous, repli de modèle, texte dégradé
`extraction/tests`	29	tableaux, lignes orphelines, modes vision/synthèse
`eiopa_agent/tests`	28	limites de Tier, arrondi, fidélité au classeur de référence, moteur réel
`climate_agent/tests`	19	SQL lecture seule, probabilités, portefeuille importé, pannes d'API

Limites ouvertes. Orchestration non intégralement portée par n8n ; corpus sans les règles de Tier et avec des diapositives-images ; évaluation du retrieval sur 13 questions seulement (indicative) et sans mesure de la qualité des réponses générées ; pas de comparaison de modèles d'embedding ; coûts API non chiffrés ; l'analyste ne dispose pas du RAG comme outil.

9 Exécution et reproductibilité

```
# 1. Service Python (RAG + agents)
cd rag_service ; python -m venv .venv ; .venv/Scripts/activate ; pip install -r requirements.txt
copy .env.example .env            # puis renseigner GEMINI_API_KEY
uvicorn main:app --port 8001
# 2. Interface Next.js (http://localhost:3000)
npm install ; npm run dev
# 3. n8n : importer n8n/solvarisk-workflow.json, l'activer (webhooks scr, report-pdf, notify)
# Tests : python -m pytest climate_agent/tests eiopa_agent/tests pipeline/tests extraction/tests
    -q
# Evaluation du retrieval : python -m eval.retrieval_eval ; python -m eval.make_report
```

Les clés API sont dans `rag_service/.env` (ignoré par git, jamais renvoyées par l'API, seul le *nom* de la variable utilisée est exposé). Le corpus se met à jour en déposant des PDF dans `rag_service/documents/` (index incrémental). Coût : modèles « flash » de Gemini avec rotation de clés ; le retrieval, le reranking et l'évaluation sont locaux.

10 Conclusion

Nous avons privilégié un principe défendable : *le LLM décide, le code calcule*. Chaque agent a un rôle mesurable, des garde-fous vérifiables et un repli déterministe ; les échecs ont fait évoluer les prompts (bibliothèque), l'architecture (agents Python) et les méthodes (stress test). La suite prioritaire : orchestration n8n du pipeline, OCR des diapositives, ajout du règlement délégué au corpus et évaluation de la qualité des réponses.